

Historia clínica soberana en Solana

Plan completo del proyecto: producto, arquitectura,
stack técnico y plan de ejecución para empezar hoy

Versión 1.0 · 3 de octubre de 2026

1. Resumen ejecutivo

Plataforma donde **el paciente es el dueño de su historia clínica**. Las clínicas y hospitales cargan los estudios después de verificar al paciente en persona. Los médicos solo pueden leerlos cuando el paciente lo aprueba, por un tiempo limitado y con registro de cada acceso. Solana es la fuente de verdad de los permisos, los prestadores habilitados y la auditoría. Los documentos médicos nunca tocan la cadena.

Qué problema resuelve

- **Paciente:** hoy su historia está dispersa en cada clínica y no sabe quién la vio. Con esto la reúne y decide quién la lee.
- **Clínica:** evita estudios duplicados, tiene trazabilidad de accesos y puede cumplir mejor con la ley de datos personales.
- **Aseguradora u obra social (etapa siguiente):** verificar si un estudio ya existe sin ver su resultado.

Por qué este enfoque y no uno más de los que ya existen

La investigación con Colosseum Copilot encontró al menos 25 proyectos de historia clínica o datos médicos en hackatones previos, casi todos con el mismo flujo (“subo un archivo cifrado y le doy permiso a una billetera”), y ninguno fue premiado. La diferenciación pesa más que la elección de herramientas. Este plan apuesta a tres palancas:

- **Vencimiento real del acceso** mediante un servicio de llaves que consulta el permiso en Solana antes de entregar cada llave.
- **Un solo cliente primero** (una prepaga u obra social, o una clínica), en lugar de “todas las clínicas”.
- **Interoperabilidad** con HL7 FHIR (en el MVP, solo una exportación de ejemplo).

Plazo: el cierre de Crypto World's Fair figura el **13/10/2026 a las 06:59 UTC**, que son las **03:59 de la madrugada del 13/10 en Argentina**. En la práctica hay que entregar antes de la noche del 12/10. Superteam Argentina puede tener plazos propios: confirmarlos en su página.

2. Alcance del MVP

Con unos 10 días hay que recortar sin piedad. El MVP tiene que mostrar de punta a punta el flujo completo, incluyendo el caso en que el paciente niega el acceso.

Dentro del MVP	Fuera del MVP (se documenta, no se construye)
Login con email o Google (Privy) para paciente, médico y personal de clínica	Cómputo confidencial con Arcium
Alta de clínicas y médicos; verificación manual por el equipo (admin)	Recuperación avanzada de cuenta con Swig (en el MVP: recuperación de Privy)
Carga de estudios por la clínica con el QR del paciente, cifrado en el navegador	App nativa con Solana Mobile (en el MVP: PWA)
Aceptar o rechazar un estudio recibido	Acceso de emergencia y contactos de confianza
Solicitud de acceso del médico y aprobación del paciente con vencimiento	Carga masiva con Merkle roots y reconciliación avanzada
Servicio de llaves que niega por defecto y entrega solo con permiso vigente	Integración real con registros de matrícula y firma digital legal
Visor del estudio con marca de agua, revocación y línea de tiempo de accesos	Borrado criptográfico automatizado (solo diseño)
Demo completa en Solana devnet	Mainnet, auditoría del programa y exportación FHIR completa

3. Actores y flujos

Actor	Rol	Cómo entra
Paciente	Dueño de su historia. Acepta estudios, aprueba o revoca accesos.	App PWA, Privy (email o Google), wallet embebida de Solana
Clínica / hospital	Emite estudios firmados. Solo ve lo que emitió ella.	Panel web, Privy, rol de personal dentro de una organización
Médico	Pide acceso y lee estudios mientras el permiso esté vigente.	Web, Privy, matrícula verificada por el admin
Admin de la plataforma	Habilita clínicas y médicos. En el MVP, el propio equipo.	Wallet de autoridad del programa
Servicio de llaves	Único componente que entrega llaves de descifrado, y solo con permiso vigente.	Backend con wallet propia (autoridad de auditoría)

Flujo A · Alta de paciente

El paciente descarga o abre la PWA, entra con Privy y se le crea una wallet embebida. Se registra su perfil en Solana (register_patient). No se guarda el DNI en la plataforma: el DNI sirve solo para que el personal de la clínica verifique a la persona en el mostrador.

Flujo B · La clínica carga un estudio

#	Paso	Dónde ocurre
1	El paciente muestra en su app un QR con su wallet pública y un código corto que vence en 2 minutos.	App del paciente
2	El personal verifica el DNI en persona y escanea el QR desde el panel. El destino es la wallet, nunca una búsqueda por DNI.	Panel de la clínica
3	El navegador genera una llave AES-256 por estudio y cifra el archivo antes de subirlo.	Navegador (WebCrypto)
4	Sube el archivo cifrado al almacenamiento con una URL firmada. La llave va al servicio de llaves, que la envuelve con la llave de la clínica y guarda solo la versión envuelta.	Storage y backend
5	La clínica firma issue_record con el hash del archivo cifrado. Queda un registro en estado <i>Pendiente</i> .	Solana
6	El paciente recibe una notificación, revisa y firma accept_record. Si no es suyo, lo rechaza y la clínica queda avisada.	App y Solana

Flujo C · El médico lee un estudio

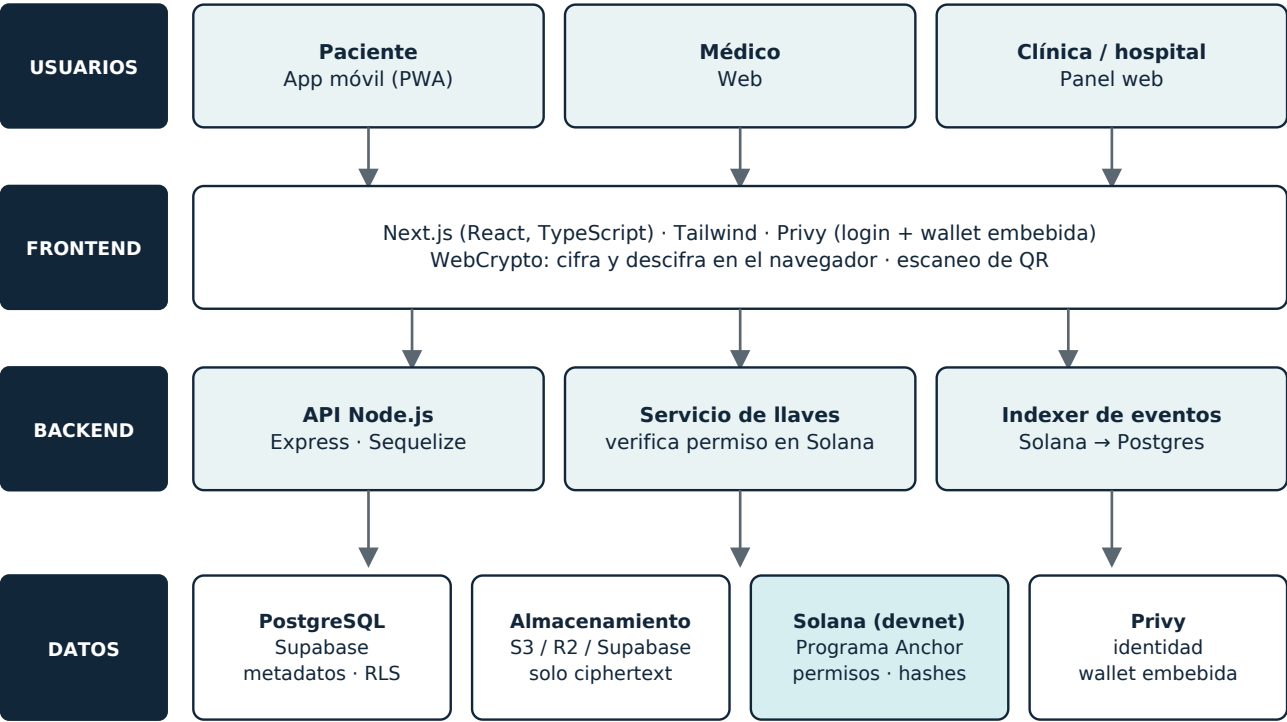
#	Paso	Dónde ocurre
1	En la consulta, el paciente le comparte un código o QR al médico. El médico pide acceso y la API notifica al paciente. La solicitud es off-chain para evitar spam y costos.	Web y API
2	El paciente ve quién pide (matrícula verificada), qué estudios y elige la duración (1 hora, 24 horas o 7 días). Firma <code>grant_access</code> .	App y Solana
3	El médico pide la llave. La API valida su sesión, lee el registro y el permiso en Solana y comprueba que estén activos y sin vencer.	Servicio de llaves
4	Si todo está bien, entrega la llave y una URL firmada de 60 segundos, y registra el acceso on-chain con <code>log_access</code> .	Servicio de llaves y Solana
5	El navegador del médico descarga, descifra y muestra el estudio en un visor sin botón de descarga y con marca de agua (nombre, matrícula y fecha).	Navegador
6	Al vencer el permiso, el servicio deja de entregar llaves. El paciente puede revocar antes con <code>revoke_access</code> .	Solana y API

Si el paciente no aprueba, no existe el permiso en la cadena y el servicio de llaves niega por defecto. El contrato no “bloquea” de forma activa: la ausencia de un permiso válido es el bloqueo. Esto conviene mostrarlo en la demo.

4. Principios de diseño

- **Nada médico en la cadena, ni siquiera cifrado.** Lo publicado en una cadena queda para siempre, y la Ley 25.326 da derecho a rectificar y suprimir datos de salud.
- **Negar por defecto.** Sin permiso explícito, vigente y firmado por el paciente, no hay llave.
- **Una llave por estudio.** Un permiso abre un estudio, no toda la historia.
- **El destino es una wallet, no un DNI.** Se vincula escaneando el QR del paciente presente. Evita mezclar homónimos o errores de tipeo.
- **Aislamiento por organización.** Cada clínica ve solo lo que emitió. Una clínica no sabe qué estudios tiene el paciente en otra.
- **Almacenamiento borrable.** Nada de Arweave o Irys para estudios. Para borrar se destruye la llave y se elimina el archivo cifrado.
- **Honestidad sobre los límites.** El servicio de llaves es un punto de confianza y un médico puede copiar lo que ya vio. Se dice en el pitch y se muestra el camino para reducirlo (Arcium o Lit).

5. Arquitectura



Los documentos médicos viajan cifrados y nunca tocan la cadena. En Solana solo hay permisos, hashes y auditoría.

Qué va en la cadena (Anchor)	Qué va en el servicio de llaves	Qué va fuera de la cadena
Identidad del paciente (perfil) Registro de prestadores verificados Por estudio: hash, referencia, emisor y estado Permisos: médico, estudio, vencimiento Eventos de auditoría de cada acceso	Verifica el permiso en Solana Entrega la llave solo si está vigente Guarda llaves envueltas, nunca en claro Firma el registro de acceso Es la pieza que hace real el vencimiento	Documentos cifrados, una llave por estudio Almacenamiento que se pueda borrar Metadatos de usuarios y organizaciones Solicitudes de acceso y notificaciones Historial indexado para la interfaz

Quién paga las comisiones de red. Para que el paciente y el médico no necesiten SOL, el backend actúa como *fee payer* y las transacciones llegan firmadas por el usuario y completadas por el servidor. Hay que limitar la tasa por usuario para evitar abuso. En el programa, el payer de las cuentas nuevas es un signer separado del dueño.

6. Stack recomendado

Capa	Recomendación	Por qué	Alternativa
Programa en Solana	Rust + Anchor	Estándar de la industria, genera IDL y tests en TypeScript	Pinocchio (más liviano, más curva)
Ciente del programa	Codama sobre el IDL de Anchor y @solana/kit	Ciente tipado generado, sin escribir instrucciones a mano	@coral-xyz/anchor
Frontend	Next.js + React + TypeScript + Tailwind + shadcn/ui, como PWA	Privy y las librerías de Solana son React primero. Una app con tres vistas por rol	Angular: hoy implicaría más trabajo con el login y la wallet
Login y wallet	Privy (email o Google, wallet embebida de Solana)	El médico y el paciente no instalan nada	Phantom con wallet-adapter
Cifrado	WebCrypto: AES-256-GCM en el navegador	Nativo, sin dependencias, el servidor nunca ve el archivo	libsodium
Backend	Node.js + TypeScript + Express + Sequelize	Stack ya conocido: evita curva de aprendizaje con poco tiempo	Fastify o NestJS, Prisma
Base de datos	PostgreSQL en Supabase con Row Level Security	Aislamiento por organización a nivel de base	Neon, Railway
Almacenamiento	Supabase Storage o Cloudflare R2 (S3)	Borrable, URLs firmadas, solo guarda ciphertext	AWS S3
Indexación	Suscripción a logs del programa (onLogs) con parser de eventos de Anchor	Sin costo extra y suficiente para la demo	Helius webhooks
RPC	Helius (plan gratuito) sobre devnet	Más estable que el RPC público	RPC público de devnet
Notificaciones	Web Push de la PWA y email transaccional	El paciente necesita enterarse para aprobar	Solo email
Hosting	Vercel (web), Railway o Render (API), Supabase (datos)	Despliegue en minutos	Fly.io
Calidad	Tests de Anchor, Vitest en la API, Playwright para el recorrido de la demo	El recorrido de la demo no puede fallar	—
Más adelante	Arcium, Lit, Swig, HL7 FHIR	Reducen la confianza en el servicio de llaves y dan recuperación e interoperabilidad	—

Antes de instalar cualquier cosa, verificar las versiones vigentes en la documentación oficial de Anchor, Codama, Privy y @solana/kit. Las APIs de estas librerías cambian rápido y este documento no fija números de versión a propósito.

7. Web3 · Programa de Solana

Cuentas (PDAs)

Cuenta	Semillas	Campos principales
Config	["config"]	admin, key_service (autoridad que firma el registro de accesos), duración máxima de permisos
Provider	["provider", authority]	authority, tipo (clínica o médico), verified, organización
PatientProfile	["patient", authority]	authority, next_record_id, creado
Record	["record", patient, record_id]	patient, issuer, content_hash (SHA-256 del archivo cifrado), storage_ref, estado (Pendiente, Aceptado, Rechazado), created_at
AccessGrant	["grant", record, doctor]	patient, doctor, record, expires_at, estado (Activo o Revocado), access_count

Instrucciones

Instrucción	Quién firma	Qué hace y qué valida
initialize_config	Admin	Crea la configuración global y fija la autoridad del servicio de llaves
register_provider / set_provider_verified	Prestador / Admin	Alta del prestador; solo el admin lo marca como verificado
register_patient	Paciente	Crea el perfil con contador de estudios
issue_record	Prestador verificado	Crea el Record en estado Pendiente. Exige prestador verificado y perfil de paciente existente
accept_record / reject_record	Paciente	Cambia el estado. Solo el dueño del perfil puede hacerlo
grant_access	Paciente	Crea el permiso. Exige Record aceptado, vencimiento futuro y menor al máximo de Config
revoke_access	Paciente	Pasa el permiso a Revocado. No se cierra la cuenta, para conservar la auditoría
log_access	Servicio de llaves	Incrementa access_count y emite el evento. Exige permiso Activo y vigente

Eventos de Anchor (emit!): RecordIssued, RecordAccepted, RecordRejected, AccessGranted, AccessRevoked y AccessLogged. El indexer los guarda en Postgres para armar la línea de tiempo. Como los logs de una transacción pueden truncarse, el contador access_count en la cuenta es la fuente de verdad.

Esqueleto de cuentas (Anchor)

```
#[account]
pub struct Record {
    pub patient: Pubkey,
    pub issuer: Pubkey,
    pub record_id: u64,
    pub content_hash: [u8; 32], // SHA-256 del archivo CIFRADO
    pub storage_ref: [u8; 32], // hash de la ruta en el storage, no la ruta
    pub status: RecordStatus, // Pending | Accepted | Rejected
    pub created_at: i64,
    pub bump: u8,
}

#[account]
pub struct AccessGrant {
    pub patient: Pubkey,
    pub doctor: Pubkey,
    pub record: Pubkey,
    pub expires_at: i64,
    pub status: GrantStatus, // Active | Revoked
    pub access_count: u32,
    pub bump: u8,
}

// grant_access: require!(expires_at > now && expires_at <= now + config.max_grant_secs)
// log_access: has_one = key_service en Config + estado Active + now < expires_at
```

Seguridad del programa

- Validar siempre signers, semillas y bump con restricciones de Anchor (has_one, seeds, constraint).
- El vencimiento se compara contra el reloj del cluster (Clock), nunca contra un valor enviado por el cliente.
- No guardar nombres, diagnósticos, DNI ni rutas legibles en ninguna cuenta.
- Probar los casos negativos: médico sin permiso, permiso vencido, permiso revocado, estudio no aceptado, prestador sin verificar, doble aceptación.
- Para el MVP, desplegar en devnet. Antes de pensar en mainnet hace falta una auditoría externa.

8. Cifrado y servicio de llaves

Elemento	Diseño
Cifrado del archivo	AES-256-GCM en el navegador con una llave aleatoria por estudio (DEK) y un IV único. Se sube solo el archivo cifrado.
Integridad	SHA-256 del archivo cifrado, guardado en el Record. El visor lo recalcula antes de descifrar.
Envoltura de la DEK	La DEK viaja por TLS al servicio de llaves, que la envuelve con una llave de la organización (KEK derivada con HKDF de una llave maestra). Se guarda solo la DEK envuelta.
Llave maestra	En el MVP, un secreto de entorno. Para producción, un KMS (AWS KMS o similar) con una llave por organización.
Acceso del paciente	El paciente puede pedir la llave de sus propios estudios aceptados: la API compara su wallet con el campo patient del Record.
Acceso del médico	Exige AccessGrant activo y vigente para ese Record y esa wallet. Se lee con compromiso <i>confirmed</i> en la demo y <i>finalized</i> en producción.
Entrega	La DEK y una URL firmada de 60 segundos, por TLS. Opcional: sellar la DEK con una clave efímera X25519 generada en el navegador del médico.
Auditoría	Cada entrega llama a log_access. Además, tabla key_releases en Postgres con usuario, estudio, hora e IP.
Borrado	Destruir la DEK envuelta y borrar el archivo cifrado. Queda en la cadena solo un hash que no revela nada.

Límites que hay que decir en voz alta: si el servicio de llaves se compromete, el atacante puede pedir llaves; y un médico que ya descifró un estudio puede copiarlo (el vencimiento frena entregas futuras, no recuerdos ni capturas). Las mitigaciones son el visor con marca de agua, TTL cortos, límites de tasa, la auditoría y, a futuro, Arcium o Lit.

9. Backend

Módulos

Módulo	Responsabilidad
auth	Verifica el token de acceso de Privy en cada request y resuelve usuario, rol y wallet
organizations	Clínicas, personal y médicos; verificación por el admin
records	URLs firmadas de subida, metadatos del estudio, estado espejo de la cadena
access	Solicitudes de acceso, notificaciones, vista de permisos del paciente
keys	Envoltura y entrega de llaves; verifica Solana; llama a log_access
tx	Arma transacciones, las completa con el fee payer y las envía
indexer	Escucha eventos del programa y los guarda para la línea de tiempo

Endpoints principales

Método y ruta	Quién	Para qué
POST /records/upload-url	Clínica	URL firmada para subir el archivo cifrado
POST /records	Clínica	Guarda DEK envuelta y metadatos; devuelve la transacción issue_record para firmar
GET /patients/me/records	Paciente	Lista de estudios y estados
POST /access-requests	Médico	Pide acceso a un estudio con el código del paciente
GET /access-requests/mine	Paciente	Solicitudes pendientes para aprobar
POST /keys/release	Paciente o médico	Entrega DEK y URL firmada si el permiso está vigente
GET /audit/:recordId	Paciente	Línea de tiempo de accesos a un estudio
POST /admin/providers/:id/verify	Admin	Marca clínica o médico como verificado

Modelo de datos (PostgreSQL)

Tabla	Campos clave
users	id, privy_id, wallet_pubkey, role, created_at (sin DNI)
organizations	id, name, kind, verified, kek_salt
staff_members	id, user_id, organization_id, role
doctors	id, user_id, license_number, verified, verified_at
records	id, organization_id, patient_wallet, record_pubkey, record_id, content_hash, storage_path, wrapped_dek, status
access_requests	id, doctor_user_id, patient_wallet, record_ids, requested_duration, status, created_at
audit_events	id, record_pubkey, doctor_wallet, kind, tx_signature, slot, created_at
key_releases	id, record_id, user_id, ip, released_at

Que no se mezclen los datos entre hospitales

- Cada fila lleva organization_id y el backend filtra siempre por la organización del usuario.

- Row Level Security en Postgres como segunda barrera: el backend ejecuta `SET LOCAL app.org_id` en cada transacción y las políticas lo exigen. Ojo: si el backend usa la clave de servicio de Supabase, RLS se saltea, así que conviene usar un rol de base sin ese privilegio.
- Rutas del almacenamiento prefijadas por organización y llaves envolventes distintas por organización.
- Un hospital no ve que el paciente tiene estudios en otro. Solo el paciente ve su historia completa.

Carga de varios estudios

Como el paciente se registra primero y se presenta en persona, una “carga masiva” se reduce a **subir varios archivos para un mismo paciente en una sola sesión**, con un `batch_id`, vista previa, claves de idempotencia (organización, id local y hash) para que reintentar no duplique, y reporte de lo cargado y lo rechazado. Cada estudio sigue pasando por la aceptación del paciente. La importación masiva desde un sistema legado con miles de pacientes queda para después, y ahí se evalúa un Merkle root por lote.

10. Frontend

Vista	Pantallas	Detalles clave
Paciente (PWA)	Inicio con estudios QR para presentarse Estudios pendientes de aceptar Solicitudes de acceso Permisos activos y revocación Línea de tiempo de accesos	Aprobación en un toque, con claridad sobre quién, qué y por cuánto tiempo. Notificaciones push. Instalable.
Clínica	Login del personal Escáner de QR del paciente Carga de uno o varios estudios Estado de cada envío	Cifrado en el navegador antes de subir. Barra de progreso por archivo. Nunca muestra datos de otras clínicas.
Médico	Solicitar acceso con código Estado de la solicitud Visor del estudio	Visor sin botón de descarga, con marca de agua y contador de tiempo restante. Mensaje claro si el paciente niega o el permiso vence.
Admin	Verificar clínicas y médicos	Pantalla mínima; se puede resolver con un script.

Librerías sugeridas

- Privy: `@privy-io/react-auth` (con soporte de Solana) para login y wallet embebida.
- Estado y datos: TanStack Query. Formularios: React Hook Form con Zod.
- QR: una librería de generación (`qrcode`) y un escáner por cámara (`@yudiel/react-qr-scanner` o similar).
- Visor: `pdf.js` para PDF e imágenes en un canvas; la marca de agua se dibuja sobre el canvas.
- PWA: Serwist o next-pwa, más Web Push con VAPID.

Si el equipo prefiere Angular, es viable, pero hay que resolver a mano el login con Privy y la firma de transacciones. Con 10 días, React reduce el riesgo.

11. Estructura del repositorio

```

historia-clinica/
├── apps/
│   ├── web/
│   └── api/
├── programs/
│   └── clinical/
├── packages/
│   ├── client/
│   └── shared/
├── docs/
├── .github/workflows/
└── pnpm-workspace.yaml

```

Next.js: paciente, clínica, médico y admin
 Node + Express: auth, records, access, keys, indexer

 Programa Anchor (Rust) + tests

 Cliente generado con Codama desde el IDL
 Tipos, esquemas Zod y utilidades de cifrado
 Este plan, decisiones y guion de la demo
 CI: anchor build/test, lint y tests de la API

12. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Vincular un estudio a la persona equivocada	Filtración de dato sensible	Destino por wallet con QR presente; el paciente acepta o rechaza; ante la duda, queda pendiente
Paciente sin celular o inconsciente	Nadie puede aprobar	Fuera del MVP: acceso de emergencia con registro y aviso, o contacto de confianza
Pérdida de la cuenta del paciente	Pierde su historia	Recuperación de Privy en el MVP; Swig o recuperación social a futuro
Médico falso	Acceso indebido con aprobación engañada	Verificación manual de matrícula; la app muestra "matrícula verificada" al paciente
Servicio de llaves comprometido	Entrega de llaves	KMS, límites de tasa, TTL cortos, auditoría; a futuro Arcium o Lit
Copia de lo ya descifrado	Vencimiento no recupera datos	Visor con marca de agua, sin descarga; decirlo en el pitch
Abuso del fee payer	Gasto de SOL y spam	Límites de tasa por usuario y cuentas con tope diario
Solicitudes de acceso como spam	Molestia al paciente	Solicitudes off-chain, solo contra un código del paciente, con límite por médico
Falla de devnet o del RPC	Se cae la demo	RPC de respaldo, video grabado del recorrido completo, datos de prueba precargados
Programa con un bug de permisos	Acceso sin aprobación	Tests negativos, revisión cruzada entre dos personas, sin mainnet sin auditoría

13. Marco legal en Argentina

Resumen para orientar, no asesoramiento legal. Conviene una consulta con un abogado antes de pasar de la demo a un piloto real.

Norma	Qué implica para el proyecto
Ley 26.529	El paciente es titular de su historia clínica. Es un buen argumento para el pitch. También pone a cargo del establecimiento la conservación de la historia (verificar el plazo vigente): la plataforma debería ser una copia cifrada bajo control del paciente, no el original.
Ley 25.326	La salud es dato sensible: hay derecho a rectificar y suprimir. Justifica no escribir nada médico en la cadena y usar almacenamiento borrrable. Consultar si corresponde registrar la base ante la autoridad de aplicación.
Ley 27.706	Programa Federal Único de Informatización y Digitalización de las Historias Clínicas, con firma digital del profesional. Una firma de billetera Solana probablemente no sea firma digital en sentido legal: validarlo.
Matrícula profesional	Para verificar médicos, evaluar el registro federal de profesionales de la salud (confirmar acceso y requisitos).

Punto de tensión a resolver: el derecho del paciente a suprimir datos choca con la obligación de conservación que tiene la clínica. El diseño propuesto separa ambos mundos: la clínica conserva su original y el paciente controla su copia cifrada y quién la lee.

14. Plan de ejecución: 10 días

Se asume un equipo chico con cuatro frentes: **Solana** (programa y tests), **Backend** (API, llaves, indexer), **Frontend** (tres vistas) y **Producto y pitch** (guion, datos de prueba, video). Una misma persona puede cubrir más de un frente. La fecha límite efectiva es la noche del 12/10.

Días	Solana	Backend	Frontend	Producto y pitch
1-2 (3 y 4/10)	Definir cuentas y reglas. Esqueleto de Anchor, config y registro de prestadores	Monorepo, base de datos, auth con Privy, entorno en Supabase	Next.js base, login con Privy, layouts por rol	Cerrar alcance, flujo y guion de demo
3-4 (5 y 6/10)	issue, accept, grant, revoke y log_access con tests negativos. Deploy en devnet	Generar cliente con Codama. Módulos tx y fee payer	Pantalla de QR y escáner. Perfil del paciente	Datos de prueba: clínica, médico y 3 estudios de ejemplo
5-6 (7 y 8/10)	Revisión cruzada del programa. Arreglos	Servicio de llaves, URLs firmadas, almacenamiento, indexer	Carga y cifrado en el navegador. Lista de estudios y aceptación	Diapositivas del pitch. Mensaje de diferenciación
7-8 (9 y 10/10)	Soporte a integración	Solicitudes de acceso, notificaciones, aislamiento por organización	Solicitud, aprobación y visor con marca de agua. Línea de tiempo	Probar el recorrido completo; anotar fallas
9 (11/10)	Congelar el programa	Límites de tasa, logs, despliegue final	Pulido, estados de error, PWA	Grabar video de respaldo de la demo
10 (12/10)	Buffer	Buffer	Buffer	Entrega antes de la noche, README y repo ordenado

Definición de terminado del MVP

- Un paciente se registra, recibe un estudio de una clínica y lo acepta.
- Un médico pide acceso, el paciente aprueba por 1 hora y el médico ve el estudio.
- El mismo médico intenta de nuevo después del vencimiento y el servicio de llaves lo niega.
- Un segundo médico sin aprobación del paciente es bloqueado.
- El paciente ve en la línea de tiempo quién accedió y cuándo, con la firma de transacción verificable en un explorador de Solana.
- Dos clínicas distintas cargan estudios para pacientes distintos y ninguna ve los datos de la otra.

15. Demo y pitch

Minuto	Qué se muestra
0:00-0:30	El problema: la historia está dispersa y nadie sabe quién la vio. La ley ya dice que el dueño es el paciente.
0:30-1:30	Clínica carga un estudio con el QR del paciente. El paciente lo acepta.
1:30-2:30	Médico pide acceso. El paciente aprueba por 1 hora. El médico lo ve.
2:30-3:15	El paciente niega a un segundo médico y el sistema bloquea. Vence el permiso del primero y también se bloquea.
3:15-4:00	Línea de tiempo con transacciones verificables. Diferenciación: el vencimiento real y un cliente concreto.
4:00-4:30	Límites asumidos con honestidad y hoja de ruta: Arcium o Lit, FHIR, emergencia, piloto con una prepaga.

Hipótesis a validar con una charla real: ¿quién paga primero, la clínica o la aseguradora? Una conversación con una prepaga u obra social vale más que otra funcionalidad. Si alguien del equipo tiene un contacto en una clínica de Jujuy, conviene pedirle 15 minutos antes del cierre.

16. Después del hackathon

- **Reducir la confianza en el servicio de llaves:** evaluar Arcium (cómputo confidencial) o Lit para que ningún servidor propio vea las llaves.
- **Recuperación y seguridad de cuenta:** Swig, recuperación social y contactos de confianza.
- **Emergencias:** acceso con doble confirmación, registro inmediato y aviso posterior al paciente.
- **Interoperabilidad:** exportar e importar recursos HL7 FHIR (Patient, DiagnosticReport, Observation).
- **Aseguradoras:** verificar la existencia de un estudio sin ver su contenido para evitar duplicados.
- **Escala:** importación masiva con Merkle roots, KMS por organización, auditoría externa del programa y paso a mainnet.
- **Validación legal:** consulta con un abogado especializado en datos de salud y firma digital.

17. Para empezar hoy

Checklist

- Crear el repositorio y el monorepo con pnpm; invitar al equipo; definir quién lleva cada frente.
- Crear cuentas en Privy, Supabase, Heliuss y el hosting elegido; guardar secretos fuera del repositorio.
- Instalar Rust, Solana CLI y Anchor; crear una wallet de desarrollo y pedir SOL de devnet.
- Escribir en una hoja las reglas de negocio del programa antes de escribir código: quién puede qué, y cuándo.
- Acordar el guion de la demo (sección 15) y los datos de prueba.

Comandos de arranque (verificar contra la documentación vigente)

```
# Monorepo
mkdir historia-clinica && cd historia-clinica && git init
pnpm init

# Programa Solana
solana config set --url devnet
solana-keygen new --outfile ~/.config/solana/id.json
solana airdrop 2
anchor init clinical          # luego mover a programs/ si se prefiere
anchor build && anchor test

# Frontend
pnpm create next-app@latest apps/web --ts --tailwind --app
pnpm --filter web add @privy-io/react-auth

# API
mkdir -p apps/api && cd apps/api && pnpm init
pnpm add express zod sequelize pg dotenv
pnpm add -D typescript tsx @types/node @types/express
```

Decisiones que el equipo debe cerrar en la primera reunión

Decisión	Recomendación de este plan
¿Quién carga el estudio y cómo se vincula?	La clínica, escaneando el QR del paciente presente
¿Solicitud de acceso on-chain u off-chain?	Off-chain; el permiso sí es on-chain
¿Quién paga las comisiones de red?	El backend como fee payer, con límites
¿Qué tipo de archivos en el MVP?	PDF e imágenes
¿Cliente objetivo para el pitch?	Una prepaga u obra social, con una clínica como piloto
¿Nombre del proyecto?	Pendiente

Este documento es un punto de partida. Lo que más reduce el riesgo en estos 10 días es tener temprano un recorrido completo, aunque sea feo, y mejorarlo después.